POLSKO-JAPOŃSKA AKADEMIA TECHNIK KOMPUTEROWYCH

SYLABUS PRZEDMIOTU

Nazwa przedmiotu:	**Konteneryzacja serwisów internetowych**
Kod przedmiotu:	KSI
Kierunek / Profil:	Informatyka / praktyczny
Tryb studiów:	niestacjonarny
Rok / Semestr:	3 / 5
Charakter:	obieralny
Odpowiedzialny:	mgr inż. Adam Urbanowicz
Wersja z dnia:	26.02.2026

1. Godziny zajęć i punkty ECTS

Wykłady	Ćwiczenia	Laboratori	Z prowa-dzącym	Praca własna	Łącznie	ECTS
16 h	—	16 h	32 h	93 h	125 h	**5**

2. Forma zajęć

Forma zajęć	Sposób zaliczenia
Wykład	Nieoceniany
Laboratorium	Zaliczenie z oceną

3. Cel dydaktyczny

Celem przedmiotu jest zapoznanie studentów z technikami konteneryzacji aplikacji webowych oraz orkiestracji kontenerów w środowiskach produkcyjnych. Student poznaje ekosystem Docker – od budowania obrazów, przez zarządzanie sieciami i woluminami, po automatyzację środowisk wielokontenerowych z użyciem Docker Compose. W dalszej części kursu student zapoznaje się z Kubernetes jako standardem orkiestracji kontenerów w chmurze, ucząc się wdrażania, skalowania i monitorowania aplikacji. Przedmiot

przygotowuje do pracy w nowoczesnych środowiskach DevOps i chmurowych (AWS, GCP, Azure).

Wersja w języku angielskim (English version)

The aim of the course is to familiarize students with techniques for containerizing web applications and orchestrating containers in production environments. Students learn the Docker ecosystem—from building images, through managing networks and volumes, to automating multi-container environments with Docker Compose. In the later part of the course, students are introduced to Kubernetes as the standard for cloud container orchestration, learning how to deploy, scale, and monitor applications. The course prepares students for work in modern DevOps and cloud environments (AWS, GCP, Azure).

4. Przedmioty wprowadzające

Przedmiot	Wymagane zagadnienia
SOP – Systemy operacyjne	Podstawowa znajomość systemów operacyjnych Linux: obsługa terminala, zarządzanie procesami, uprawnienia, sieci.
PAI – Programowanie aplikacji internetowych	Umiejętność tworzenia aplikacji webowych w Node.js lub innym frameworku backendowym; znajomość protokołu HTTP.
TIN – Technologie internetu	Podstawowa znajomość architektury aplikacji webowych po stronie klienta i serwera.

5. Treści programowe

1. Wirtualizacja a konteneryzacja – historia, różnice i zastosowania. Architektura Dockera: daemon, klient, obrazy, kontenery, rejestry. Instalacja i podstawowe polecenia CLI.
2. Dockerfile – budowanie własnych obrazów. Instrukcje: FROM, RUN, COPY, ADD, ENV, ARG, EXPOSE, WORKDIR, ENTRYPOINT, CMD. Warstwy obrazu i cache. Plik .dockerignore.
3. Zaawansowane techniki budowania obrazów: multi-stage build, minimalizacja rozmiaru, obrazy bazowe (Alpine, Distroless), skanowanie podatności (Trivy). Publikowanie w Docker Hub i prywatnym rejestrze.
4. Sieci Dockera: tryby bridge, host, none, overlay, macvlan. Komunikacja między kontenerami. Tworzenie własnych sieci. Resolucja DNS wewnątrz sieci Dockera.
5. Persystencja danych: volumes zarządzane, bind mounts, tmpfs. Cykl życia voluminów. Współdzielenie danych między kontenerami. Backup i przywracanie danych.

6. Docker Compose – definiowanie środowisk wielousługowych. Składnia pliku docker-compose.yml: services, networks, volumes, environment, depends_on, healthcheck. Polecenia compose up/down/logs/exec.
7. Zaawansowane Docker Compose: profile, rozszerzenia YAML (anchors), override files, skalowanie usług. Środowiska developerskie vs. produkcyjne.
8. Wprowadzenie do orkiestracji kontenerów. Architektura Kubernetes: control plane (API Server, etcd, Scheduler, Controller Manager) i węzły robocze (kubelet, kube-proxy). Instalacja lokalna: Minikube, k3s.
9. Podstawowe obiekty Kubernetes: Pod, ReplicaSet, Deployment. Manifesty YAML. Kubectl: get, describe, apply, delete, logs, exec. Namespaces.
10. Serwisy Kubernetes: ClusterIP, NodePort, LoadBalancer, ExternalName. Ingress i Ingress Controller (NGINX). Eksponowanie aplikacji na zewnątrz klastra.
11. Konfiguracja i sekrety w Kubernetes: ConfigMap, Secret (Opaque, TLS, docker-registry). Przekazywanie konfiguracji do podów przez zmienne środowiskowe i woluminy.
12. Persystencja w Kubernetes: PersistentVolume (PV), PersistentVolumeClaim (PVC), StorageClass. StatefulSet vs. Deployment dla aplikacji stanowych.
13. Skalowanie i aktualizacje: Horizontal Pod Autoscaler (HPA), resource requests i limits, rolling update, rollback, PodDisruptionBudget. Podstawy Vertical Pod Autoscaler.
14. Bezpieczeństwo kontenerów i klastra: zasada najmniejszych uprawnień, SecurityContext, NetworkPolicy, RBAC (Role, ClusterRole, RoleBinding), zarządzanie sekretami z Vault.
15. CI/CD w ekosystemie kontenerowym: budowanie i publikowanie obrazów w GitHub Actions / GitLab CI. Strategie wdrożeń: rolling, blue-green, canary. Helm – pakietowanie i wdrażanie aplikacji. Przegląd usług zarządzanych: AWS ECS/EKS, GCP GKE, Azure AKS.

6. Przedmiotowe efekty uczenia się

- Student zna i rozumie koncepcję wirtualizacji na poziomie systemu operacyjnego oraz różnice między maszynami wirtualnymi a kontenerami pod względem izolacji, wydajności i przenośności.
- Student zna i rozumie architekturę Dockera: daemon, klient, rejestr obrazów (Docker Hub, prywatny registry), warstwy obrazu i mechanizm union filesystem.
- Student zna i rozumie cykl życia kontenera oraz zasady tworzenia efektywnych obrazów Dockera z użyciem instrukcji Dockerfile: FROM, RUN, COPY, ENV, EXPOSE, ENTRYPOINT, CMD.
- Student zna i rozumie mechanizmy sieciowe Dockera (bridge, host, overlay, macvlan) oraz sposoby persystowania danych (volumes, bind mounts, tmpfs).
- Student zna i rozumie przeznaczenie i budowę pliku docker-compose.yml: definiowanie usług, sieci, woluminów, zmiennych środowiskowych i zależności między serwisami.
- Student zna i rozumie architekturę Kubernetes: komponenty warstwy sterowania (API Server, etcd, Scheduler, Controller Manager) oraz węzłów roboczych (kubelet,

kube-proxy, container runtime).
- Student zna i rozumie podstawowe obiekty Kubernetes: Pod, ReplicaSet, Deployment, Service (ClusterIP, NodePort, LoadBalancer), ConfigMap, Secret, PersistentVolume, PersistentVolumeClaim, Ingress.
- Student zna i rozumie zasady skalowania aplikacji w Kubernetes: skalowanie manualne, Horizontal Pod Autoscaler (HPA) oraz rolling update i rollback.
- Student zna i rozumie zagadnienia bezpieczeństwa kontenerów: zasadę najmniejszych uprawnień, skanowanie obrazów pod kątem podatności (Trivy, Snyk), zarządzanie sekretami oraz polityki sieciowe (NetworkPolicy).
- Student zna i rozumie podstawy CI/CD w kontekście konteneryzacji: budowanie i publikowanie obrazów w potoku CI (GitHub Actions, GitLab CI), strategie wdrożeń (rolling, blue-green, canary).

Umiejętności

- Student potrafi napisać poprawny plik Dockerfile dla aplikacji webowej (Node.js, Python lub Java), stosując best practices: multi-stage build, minimalizację warstw, użytkownik non-root, .dockerignore.
- Student potrafi zbudować, otagować i opublikować obraz Docker w rejestrze Docker Hub oraz w prywatnym rejestrze.
- Student potrafi skonfigurować plik docker-compose.yml dla środowiska wielokontenerowego (aplikacja webowa + baza danych + cache), uruchamiać, zatrzymywać i debugować usługi.
- Student potrafi skonfigurować sieci i woluminy Dockera w celu zapewnienia izolacji i persystencji danych między kontenerami.
- Student potrafi wdrożyć aplikację na lokalny klaster Kubernetes (Minikube lub k3s), tworząc manifesty YAML dla obiektów Deployment, Service i Ingress.
- Student potrafi skonfigurować ConfigMap i Secret w Kubernetes oraz przekazywać zmienne środowiskowe i pliki konfiguracyjne do podów.
- Student potrafi przeprowadzić rolling update i rollback wdrożenia w Kubernetes oraz monitorować stan klastra poleceniami kubectl (get, describe, logs, exec, top).
- Student potrafi skonfigurować Horizontal Pod Autoscaler i zweryfikować działanie automatycznego skalowania pod obciążeniem.
- Student potrafi skonfigurować prosty potok CI/CD (GitHub Actions) automatycznie budujący i publikujący obraz Docker po każdym push do repozytorium.
- Student potrafi zidentyfikować i usunąć podstawowe problemy z kontenerami (crashloopbackoff, brak zasobów, błędy konfiguracji sieci) przy użyciu narzędzi diagnostycznych.

Kompetencje społeczne

- Student jest gotów do stosowania podejścia Infrastructure as Code, traktując pliki konfiguracyjne (Dockerfile, docker-compose.yml, manifesty Kubernetes) jako integralną część kodu źródłowego przechowywanego w systemie kontroli wersji.
- Student jest gotów do krytycznej oceny zasadności konteneryzacji w danym projekcie, uwzględniając koszty utrzymania infrastruktury, złożoność operacyjną i wymagania

bezpieczeństwa.
- Student jest gotów do samodzielnego pogłębiania wiedzy z zakresu technologii chmurowych i DevOps, śledząc dokumentację, blogi techniczne i społeczność open source (CNCF).

7. Kryteria oceny

- Laboratorium – zaliczenie z oceną:
- Ocena ciągła na podstawie sprawozdań z zajęć laboratoryjnych (40% oceny końcowej laboratorium).
- Projekt końcowy: skonteneryzowanie wielokomponentowej aplikacji webowej (frontend, backend, baza danych) z użyciem Docker Compose oraz wdrożenie jej na lokalny klaster Kubernetes z konfiguracją Ingress i zmiennymi środowiskowymi (60% oceny końcowej laboratorium).
- Warunkiem zaliczenia laboratorium jest oddanie wszystkich sprawozdań oraz obrona projektu końcowego.
- Ocena końcowa przedmiotu: średnia ważona oceny z egzaminu (60%) i oceny z laboratorium (40%).

8. Metody dydaktyczne

wyklad:
- Wykład z prezentacją multimedialną, demonstracje na żywo, analiza przypadków przemysłowych, omawianie dokumentacji technicznej Docker i Kubernetes.

laboratorium:
- Ćwiczenia praktyczne: budowanie, uruchamianie i orkiestracja kontenerów z użyciem Docker, Docker Compose oraz Kubernetes; realizacja projektu końcowego obejmującego pełny cykl konteneryzacji aplikacji webowej.

9. Literatura

Podstawowa:

- N. Mouat: "Docker. Projektowanie i wdrażanie aplikacji", Helion, Gliwice 2016 (Using Docker, O'Reilly 2015).
- B. Burns, J. Beda, K. Hightower: "Kubernetes. Tworzenie niezawodnych systemów rozproszonych", Helion, Gliwice 2023 (Kubernetes: Up and Running, 3rd ed., O'Reilly 2022).
- dokumentacja oficjalna Docker: docs.docker.com.
- dokumentacja oficjalna Kubernetes: kubernetes.io/docs.

Uzupełniająca:

- B. Kane: "Docker w praktyce", Helion, Gliwice 2019 (Docker in Practice, 2nd ed., Manning 2018).
- M. Luksa: "Kubernetes w akcji", Helion, Gliwice 2019 (Kubernetes in Action, Manning 2018).
- M. Kleppmann: "Przetwarzanie danych w dużej skali. Niezawodność, skalowalność i łatwość konserwacji systemów", Helion 2021.
- The Twelve-Factor App: 12factor.net – metodologia budowania aplikacji gotowych do konteneryzacji.